

Summary and Analysis: SimCLR and MoCo

Moataz Hamza

Academic College of Tel Aviv-Yaffo

Abstract

As part of a seminar on innovations and trends in machine learning taught by Prof. Raid Saabni, I was asked to explore and deliver a lecture on the subject of self-supervised learning, specifically the *contrastive learning* approach. In the lecture I presented the *SimCLR* and *MoCo* papers. I decided to re-read the papers and write this analysis for two reasons: the first one being that after the presentation I felt that there were some small gaps in my understanding of the papers. And the second reason is because I'm planning on re-implementing and comparing these two different techniques by myself, trying different architectures and augmentations.

1 Shared Problem and Approach

Both papers address the same problem: self-supervised learning of useful visual representations. Because not every problem has a large labeled data set which we can use to train our model on, self-supervised learning is a way of attaining knowledge from other tasks that we create, and have large datasets for, and later use that knowledge to solve our original problem with some fine-tuning on the original much smaller dataset.

For e.g. a huge data set of images can be used to create input-label pairs where the input is a *rotated* version of an image from the dataset, and the label is the degree of rotation, and then our task is to predict the rotation applied in this image. After that we can take the pretrained model and finetune it on the small data set of our original problem.

In both papers the same approach was used to learn, and that is *Contrastive Learning*. In Contrastive learning, our encoder learns how to represent the images by creating similar representations for similar images, and creating dissimilar representations for dissimilar images.

In the MoCo paper, they claimed that unsupervised learning of representations was not as successful in computer vision as it is successful in NLP. They criticized previous method for two things:

1. That the methods that used contrastive learning without having a memory bank were limited by batch sizes, because batch sizes control the number of negative samples
2. The methods that did use a memory bank have a consistency problem, as keys encoded early in training, and are saved in the memory bank, become outdated as the encoder continues to change.

In SimCLR they critiqued the existing approaches for self-supervised learning in computer vision. They claimed that the generative approach was too expensive, the discriminative approach relied on pretext tasks that limited the generality of the representations. And they also claimed that the previous frameworks for self-supervised

learning were either reliant on architecture changes or memory banks, which made these frameworks less simple.

2 Different Views

2.1 Dictionary Look-up

MoCo viewed contrastive learning as a dictionary look-up problem, where you have the first view of a certain image x_q as the query, and the encoder is trained to "look" for the second view of that image x_k , which will be that query's key, among other keys in the dictionary. So the encoder gets trained to give the query a similar visual representation to the key, which helps finding that correct key in the dictionary.

2.2 Positive and Negative Pairs

In SimCLR they did not look at pairs in the data as query-key, but as positive pairs, two augmented versions of the same image, or else a negative pair. So the encoder was trained to maximize the similarity between a positive pair, and at the same time minimize the similarity between the negative pairs.

3 Loss Function

The loss function in both papers was based on the *InfoNCE* loss. The main difference between them is the similarity metric they used. In MoCo they used the standard dot product, but in SimCLR they used the normalized version which is the *cosine similarity*.

$$L_q = -\log \frac{\exp(q \cdot k_+ / \tau)}{\sum_{i=1}^K \exp(q \cdot k_i / \tau)} \quad (\text{MoCo})$$

$$L_{ij} = -\log \frac{\exp(\text{sim}(z_i \cdot z_j) / \tau)}{\sum_{k=1(\neq i)}^{2N} \exp(\text{sim}(z_i \cdot z_k) / \tau)} \quad (\text{SimCLR})$$

To minimize this loss function we would need to maximize the numerator, meaning we would need to maximize the similarity between the positive pair, or the query-key pair. We would also need minimize the denominator, meaning we would need to minimize similarity between the query and all the negative keys.

We can notice that there are a couple more differences, e.g. the loss in MoCo is defined for every query, and in SimCLR it is defined for each positive pair twice, once where i is treated as the query and once where j is treated as such. This shows the symmetry in how the two augmented views in an image are treated in SimCLR, where as in MoCo they have different roles.

In the SimCLR paper they pointed out that this is basically the cross-entropy loss applied on a softmax, where the correct class is the positive pair.

4 Key Components

4.1 MoCo's Queue and Momentum Encoder

As an improvement to the weaknesses in previous methods, MoCo introduced their improved "memory bank" implemented as a queue, with the keys going into it encoded by their *Momentum encoder*. Some of the earlier methods did not use a memory bank at all, which means that they had to get their negative samples from the current batch, and that limits the number of negative samples, and previous work showed that the contrastive loss works better with more negative samples. So that's why in MoCo they emphasize the importance of decoupling the batch size from the negative samples count, because the batch size is bounded by physical limitations, by using a memory bank.

But they also made an improvement to their memory bank by trying to keep the keys as consistent as possible with keys encoded by future encoders. That's where the momentum encoder comes in. Their first try was having the same encoder for the queries and keys, but the problem is that this encoder gets updated and changed quickly, which makes the keys in the dictionary outdated representations for future batches and the results show that this affected the performance. So what they did is they presented this new encoder for the keys, that updates similarly to the queries encoder, but much more slowly, using a moving average:

$$\theta_k = \theta_k \cdot \beta + \theta_q \cdot (1 - \beta)$$

Where θ_k are the parameters of the momentum encoder, θ_q are the parameters of the queries encoder, and β is the parameter that controls the speed of the updates, the larger the slower the momentum encoder updates.

4.2 SimCLR's Data Augmentation and Projection Head

Unlike MoCo, in the SimCLR paper they emphasize the importance of using different augmentations and displayed how the use of multiple augmentations improved performance, and even MoCov2 showed that later. In SimCLR they put data augmentations in two categories, spatial transformations like cropping and resizing, and appearance transformations like color distortions and blurs. They even tested out different combinations of augmentations and compared them with single augmentations and found out that using a combination of augmentations always enhances performance, particularly the combination of a spatial and an appearance transformation, especially cropping and resizing and color distortion. That's why they used a combination of cropping and resizing, color distortion and gaussian blur, ordered randomly, to make their augmented data.

They also tested out adding three different types of projection heads right after the encoder's output:

1. Identity function
2. Linear projection head
3. Non-linear projection head

with the non-linear variant outperforming the others every single time.

They also show that even when using a non-linear projection head g it's better to use the output of the layer before the projection head h . They hypothesized that this happens because the projection head decides to ignore some information like the colors or the rotation to solve the contrastive task, with the goal to make similar images closer in representation. And they tested it out by training a linear classifier to predict augmentations performed on some images, once on top of h and once on top of g , with the one on top of h outperforming. Meaning that h has more info on the image, and therefore is a better representation.

5 More Details

Both papers experimented with the same networks as encoder, which are different versions of ResNet50. SimCLR presented lots of experiments, for e.g. they showed that self-supervised learning benefits from larger models and training time, more than supervised learning. They also showed how much their framework benefited from larger batch sizes. SimCLR outperformed MoCo, even with smaller batch sizes, but later MoCov2 came out and beat SimCLR using the two ideas SimCLR presented on data augmentations and the projection head. The main evaluation method used was the linear evaluation protocol: training a linear classifier on top of the pretrained encoder, which will be frozen, on ImageNet.

	epochs	batch	ImageNet acc.
MoCo	200	256	60.6
SimCLR	200	256	61.9
SimCLR	200	8192	66.6
MoCo v2	200	256	67.5

6 Strengths and Weaknesses

SimCLR's main strength is that it was true to its name, it is simple, though it needed a large batch size to perform well. MoCo, by contrast, did not, and with slight changes introduced in MoCoV2 it eventually outperformed SimCLR with smaller batch sizes. The trade-off is added complexity: MoCo requires a memory bank.

7 Critique

For both MoCo and SimCLR I would have liked to see them experimenting with different encoder architectures.

Unlike in MoCo, SimCLR did not experiment with other computer vision tasks like object detection, and also I'm still not sure that I understand how exactly they applied their augmentations, they said that they were using 3 types of them applied sequentially, and in their algorithm it says that the augmentations are chosen randomly, so is it the order that is random? and in MoCo they did not even say what augmentations they used.

Reading these two papers felt like two different experiences, even though they are solving the same problem in a similar way. MoCo felt like "Okay, this is our new technique and these are the results that show that it works". But for SimCLR it felt like "Okay, we tried this, and it works, and here is why it works".

8 The False Negative Problem

During my presentation, the professor asked a key question that I did not have an answer for at the time, and it annoyed me that it had not occurred to me beforehand: *"What if there is a picture of another dog in the dataset? Wouldn't that picture be treated as a negative sample, forcing the model to push their representations apart even though both are dogs and should share structural similarity?"*

This question highlights a fundamental challenge in self-supervised contrastive learning known as the **False Negative problem**. Because standard frameworks like SimCLR and MoCo operate entirely without class labels, they treat all samples in a mini-batch or dictionary queue (other than the augmented view of the same source image) as negative pairs. When two distinct images happen to share the same semantic category (e.g., two different dogs), forcing their representations apart injects noisy gradients into the optimization process.

To address this limitation, several works explicitly attempt to identify and mitigate false negatives during pre-training. Two key papers that tackle this issue, which I plan to read and analyze next, are:

- *Incremental False Negative Detection for Contrastive Learning* by Chen et al. [3], which proposes an incremental sampling strategy to detect and remove potential false negatives dynamically throughout training.
- *Boosting Contrastive Self-Supervised Learning with False Negative Cancellation* by Huynh et al. [4], which introduces a false negative cancellation technique (FN-SSL) to re-weight or remove conflicting negative pairs based on feature similarity.

References

- [1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020.
- [2] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9729–9738, 2020.
- [3] Tsai-Shien Chen, Wei-Chih Hung, Hung-Yu Tseng, Shao-Yi Chien, and Ming-Hsuan Yang. Incremental false negative detection for contrastive learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3522–3531, 2021.
- [4] Tri Huynh, Simon Kornblith, Matthew R. Walter, Michael Maire, and Maryam Khademi. Boosting contrastive self-supervised learning with false negative cancellation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2785–2795, 2022.